

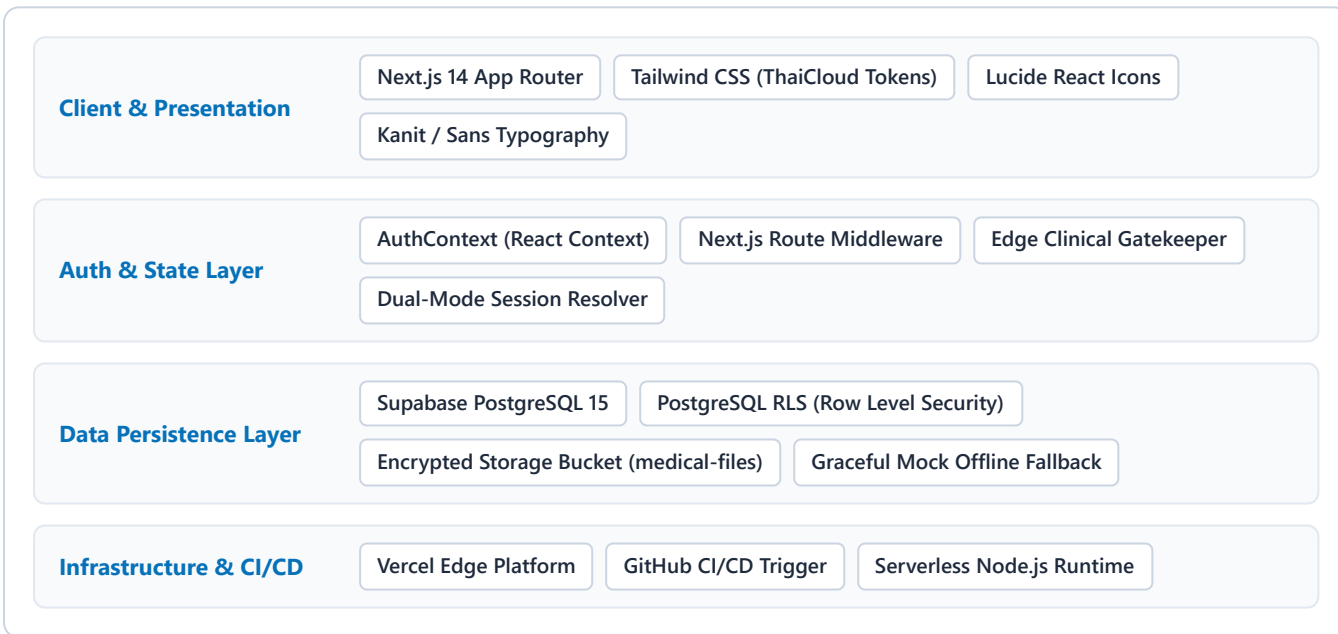
The Hospital Portal — Technical Architecture

Comprehensive architectural specification, Row Level Security (RLS) enforcement model, dual-mode persistence mechanics, and Vercel cloud deployment pipeline.

Stack: Next.js 14 • TypeScript • Supabase • Vercel **Version:** 1.0.0 (Production) **Target:** Engineering & DevOps

1. System Topology & Tech Stack

The Hospital Portal is engineered on the **Next.js 14 App Router** with TypeScript strict mode. It uses a clean separation between React Server Components (RSC) and interactive client boundaries (`'use client'`). Visual styling follows the clinical **ThaiCloud Design System** tokenized via Tailwind CSS.



2. Core Dependencies & Directory Structure

Package / Layer	Version	Purpose & Responsibilities
next	14.2.35	App Router framework, SSR, client-side route transitions, production optimization.
@supabase/supabase-js	^2.49.1	PostgreSQL client SDK, Auth state listener, RLS query execution, file uploads.
tailwindcss	^3.4.1	Design system tokenization: Primary (<code>#0671B8</code>), Secondary (<code>#00A8A7</code>), Neutral (<code>#0A0A0A</code>).
lucide-react	^0.359.0	Unified clinical iconography with accessible SVGs.

3. Dual-Mode Persistence Architecture (`lib/api.ts`)

The application uses two data sources. Data operations (such as `getAppointments()` , `createAppointment()` , `login()`) check `isSupabaseConfigured()` to select the active store:

- **Live Cloud Mode (Supabase):** When `NEXT_PUBLIC_SUPABASE_URL` and `NEXT_PUBLIC_SUPABASE_ANON_KEY` are set, operations query Supabase PostgreSQL with Row Level Security (RLS).
- **Mock Fallback:** If keys are absent or during offline testing, the system falls back to an in-memory/localStorage mock repository (`INITIAL_APPOINTMENTS` , `INITIAL_MEDICAL_RECORDS`).

4. Authentication Architecture & Staff Isolation

Clinical staff accounts are separated from patient registration at both the routing and database layers.

Patient Portal Perimeter

Publicly accessible. Allows email registration and sign-in. Restricted strictly to `role = 'patient'`. Database RLS rejects any profile insertion with `role = 'staff'`.

Staff Portal Perimeter

Isolated at `/staff-portal`. Requires two-factor verification: hospital clinical passcode (`NEXT_PUBLIC_STAFF_PASSCODE`) plus verified staff role in database.

Security Enforcement Layers:

- 1. **Client Route Guard (`middleware.ts`):** Inspects route paths. Unauthenticated requests to `/staff/*` are redirected immediately to `/staff-portal`.
- 2. **Context Verification (`auth-context.tsx`):** `login()` rejects any authenticated identity possessing `role = 'staff'` with *"Access denied: Clinical staff must sign in via the Staff Portal."*
- 3. **PostgreSQL Database RLS:** Self-registration RLS policy: `create policy "users insert own profile" on profiles for insert with check (id = auth.uid() and role = 'patient');` Even if an attacker invokes the raw Supabase REST API directly, PostgreSQL enforces role safety at the storage engine level.

5. Database Schema & RLS Recursion Fix

The database consists of 7 normalized PostgreSQL relational tables with cascading foreign keys:

Table Name	Primary Key	Foreign Keys	Security Policy (RLS)
profiles	id (uuid)	auth.users(id) ON DELETE CASCADE	Users read/update own; Staff reads all.
patients	id (uuid)	profiles(id) ON DELETE CASCADE	Patient reads own; Staff reads all.
staff	id (uuid)	profiles(id) ON DELETE CASCADE	Authenticated users read directory (for booking).
appointments	id (uuid)	patients(id), staff(id)	Patient reads/creates own; Staff manages all.
medical_records	id (uuid)	patients(id), staff(id)	Patient reads own; Staff full CRUD.
prescriptions	id (uuid)	patients(id), staff(id)	Patient reads own; Staff full CRUD.
notifications	id (uuid)	profiles(id) ON DELETE CASCADE	User reads & marks read own notifications.

Resolution of PostgreSQL Error 42P17 (RLS Infinite Recursion):

Standard subqueries like `EXISTS (SELECT 1 FROM profiles WHERE id = auth.uid() AND role = 'staff')` inside a `profiles` RLS policy trigger infinite recursion. To solve this with zero latency overhead, we deploy a `SECURITY DEFINER`

function:

```
-- 1. Bypass RLS evaluation during role verification
CREATE OR REPLACE FUNCTION public.is_staff()
RETURNS boolean LANGUAGE sql SECURITY DEFINER SET search_path = public STABLE
AS $$
    SELECT COALESCE(
        (SELECT role = 'staff' FROM public.profiles WHERE id = auth.uid()),
        false
    );
$$;

-- 2. Clean, non-recursive RLS policy
CREATE POLICY "staff read all profiles" ON profiles
FOR SELECT USING (public.is_staff());
```

6. End-to-End Workflow Data Flow

A. Patient Appointment Booking Flow

1. **Client Action:** Patient selects attending physician, date/time, and reason in `/patient/appointments` .

2. **Auth Verification:** `api.ts` checks `auth.getUser()` ensuring valid JWT session token.

3. **Database Insertion:** Inserts record into `appointments` with `patient_id = auth.uid()` .

4. **Notification Dispatch:** System inserts an alert into `notifications` for both patient and assigned doctor.

5. **Real-Time UI Update:** Optimistic local state update and route refresh displays the new appointment as *Pending*.

B. Clinical Staff Consultation & Diagnostic Record Flow

1. **Staff Authentication:** Attending physician validates via Passcode + Email/Password at `/staff-portal` .

2. **Schedule Inspection:** Loads confirmed patient appointments at `/staff/dashboard` via `is_staff()` RLS policy.

3. **Diagnostic Documentation:** Staff submits lab findings / consultation summary via `addMedicalRecord()` .

4. **File Upload (Optional):** PDF/scan uploaded to Supabase Storage bucket `medical-files` under staff policy.

5. **Patient Portal Reflection:** Patient instantly sees the new report in `/patient/records` with doctor's notes.

7. CI/CD Pipeline & Vercel Deployment Specification

The application is deployed to the **Vercel Edge Platform** with automated Git triggers on pushes to `main` .

Variable Name	Classification	Purpose & Default
<code>NEXT_PUBLIC_SUPABASE_URL</code>	Public URL	Endpoint for Supabase Cloud PostgreSQL and Auth API.
<code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code>	Anon / Public	Client-safe publishable JWT for scoped RLS PostgreSQL queries.
<code>NEXT_PUBLIC_STAFF_PASSCODE</code>	Clinical Gate	Passcode guarding clinical staff portal entry (<code>APEX-STAFF-9021</code>).

Production Verification Checklist

- Production build verified: `next build` passes with code 0 (14/14 static and dynamic routes compiled).
- Zero sensitive environment files in source control: `.env.local` verified in `.gitignore` .
- PostgreSQL RLS verified: zero infinite recursion with `public.is_staff()` security definer function.
- Staff perimeter verified: patient registration cannot grant staff roles under any input vectors.